Reviewer Pack — Minimal Rank of Geometric Invariants in Motivic Homology ove…

Entry 04888a3c8fe7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 03:05:02 UTC

Minimal Rank of Geometric Invariants in Motivic Homology over Tseitin Circuit Width

Entry ID: `04888a3c8fe7` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 03:05:02 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Motivic Homology) **Field B** (complexity object): Complexity Theory: Tseitin Circuit Width

Statement:

> ['For a given Tseitin circuit C with width w, the minimal rank of the motivic homology group H^1(C) is at least 2^(w/2).', 'This implies that for circuits with width w, the Resolution proof length is at least 2^(w/2).', 'For all Tseitin circuits C with width less than or equal to 40, this conjecture can be tested within 240 seconds using Python.']

Rationale (proposer's reasoning):

> ['Motivic homology provides a rich algebraic structure that could potentially capture the complexity of Tseitin circuits.', 'Previous work has shown connections between motivic homology and circuit complexity, suggesting that it may be useful for lower bounds.', 'This conjecture aims to establish a direct link between geometric invariants from algebraic geometry and complexity theory.']

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `08b04765fed476a7`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a Tseitin circuit C with width up to 40, H^1(C)'s rank is considered supported if the mean rank across all 30 seeds is at least 2^(w/2) and no seed has a rank less than 2^(w/2). It is falsified if any seed produces a rank less than 2^(w/2) or if the mean rank is less than 2^(w/2).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "motivic homology" AND "Tseitin circuit width" - "minimal rank" IN "motivic homology" AND Tseitin circuit" - "Resolution proof length" AND 2^(w/2) AND Tseitin circuit

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        m, n = len(matrix), len(matrix[0])
        rank = 0
        for j in range(n):
            i_max = rank
            for i in range(rank, m):
                if abs(matrix[i][j]) > abs(matrix[i_max][j]):
                    i_max = i
            if matrix[i_max][j] == 0:
                continue
            matrix[rank], matrix[i_max] = matrix[i_max], matrix[rank]
            for i in range(m):
                if i != rank:
                    factor = -matrix[i][j] / matrix[rank][j]
                    for k in range(n):
                        matrix[i][k] += factor * matrix[rank][k]
            rank += 1
        return rank

    def tseitin_circuit(width):
        n = width + 2
        variables = list(range(1, n))
        clauses = []
```

```
        # Add clauses for each variable
        for i in range(n - 2):
            clauses.append([variables[i], variables[n - 2]])

        # Add clauses for the OR gate
        for i in range(n - 3):
            clauses.append([-variables[i], -variables[n - 3]])

        # Add clauses for the AND gate
        for i in range(n - 4):
            clauses.append([variables[i], variables[n - 4]])

        return clauses

    def motivic_homology_rank(clauses):
        n = len(clauses)
        m = len(variables) + n
        matrix = [[0] * (m + 1) for _ in range(m)]

        for i, clause in enumerate(clauses):
            for var in clause:
                if var > 0:
                    matrix[i][var - 1] = 1
                else:
                    matrix[i][-var - 1] = 1

        return gaussian_elimination(matrix)

    width = random.randint(5, 40)
    circuit = tseitin_circuit(width)
    rank = motivic_homology_rank(circuit)

    metric_name = "motivic_homology_rank"
    metric_value = rank
    instances_tested = 1
    conjecture_holds = rank >= 2 ** (width / 2)
    counterexample = "" if conjecture_holds else f"Rank {rank} is less than 2^{width/2}"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 30))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
```

```
    mean_rank = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
    elif any(not result["conjecture_holds"] for result in results) or support_fraction < 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Rank less than 2^(w/2)\" first_failing_seed={fi
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b8537a1e.py", line 96, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b8537a1e.py", line 75, in run_trial
    rank = motivic_homology_rank(circuit)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b8537a1e.py", line 61, in motivic_homol
    m = len(variables) + n
            ^^^^^^^^^
NameError: name 'variables' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that it was unable to complete the computation and verify the conjecture. | next: Investigate the error in the test code to identify the cause of the crash and correct it. Once fixed, re-run the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	?	?	0	0	15141
2	propose	ollama_remote	glm4:latest	0	0	24302

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	preregistration	ollama_remote	glm4:latest	0	0	11343
4	novelty	ollama_remote	glm4:latest	0	0	4958
5	novelty	ollama_remote	glm4:latest	0	0	5632
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13085
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9001
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7136
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10243
10	judge	ollama_remote	glm4:latest	0	0	15964

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 116805 ms total latency. Provider mix: {'?': 1, 'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/04888a3c8fe7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/04888a3c8fe7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/04888a3c8fe7.tar.gz` (if generated)